

# ECE-UY 3064: Feedback Control — Experiment 4

## Digital Gain-Lead-Lag Compensation for DC Motor

MECH/EE Lab with NI-USB6002

Demarce Williams  
DSW9740  
NYU Tandon School of Engineering

May 1, 2026

## 1 Introduction

The objective of this lab is to design and implement a digital gain–lead–lag compensator to control the position of a DC motor. This involves designing a continuous-time controller, converting it to a discrete form using the Tustin transformation, and implementing it in C++ as a real-time control algorithm.

The controller combines:

- **Lead compensation** to improve speed and stability
- **Lag compensation** to reduce steady-state error
- **Gain adjustment** to tune overall performance

The system is then tested experimentally to verify accurate and stable motor control.

## 2 Background and Theory

The objective of this section is to provide the theoretical foundation for designing and implementing a digital **gain-lead-lag compensator** to control the position of a DC motor. This involves understanding the continuous-time behavior of each compensator element and the mathematical process of discretizing them for software implementation.

### 2.1 Gain-Lead-Lag Compensator Overview

A gain-lead-lag compensator combines three distinct control elements to achieve a balance between stability, speed, and accuracy. The general continuous-time transfer function for this controller is defined as:

$$G_c(s) = K \frac{p_d s + z_d}{z_d s + p_d} \frac{s + z_g}{s + p_g}$$

where  $K$  is the gain, the subscript “ $d$ ” represents the lead parameters, and “ $g$ ” represents the lag parameters.

**Lead Compensation:** Designed to introduce a **phase lead**, which increases the system's **phase margin** and crossover frequency. This improves the transient response speed but can amplify high-frequency noise. The maximum phase increase ( $\phi_{max}$ ) occurs at the frequency  $\omega_{max} = \sqrt{z_d p_d}$ .

**Lag Compensation:** Functions similarly to a PI controller at low frequencies, providing **high DC gain** to significantly reduce **steady-state tracking error**. It is designed to attenuate high-frequency gain so that the system crosses the zero dB line before the phase reaches  $-180^\circ$ , thus maintaining stability.

**Gain Compensation:** Simple proportional feedback used to reduce steady-state error and increase bandwidth. However, excessive gain can reduce the phase margin and potentially **destabilize the system**.

## 2.2 Controller Discretization

Because the control law is implemented on a digital computer, the continuous-time transfer functions must be converted into discrete-time equivalents. This experiment primarily utilizes the **bilinear (Tustin) transformation**.

The mapping from the  $s$ -plane to the  $z$ -plane is given by the approximation:

$$s = \frac{2}{T} \frac{1 - z^{-1}}{1 + z^{-1}}$$

where  $T$  is the sampling period. This transformation is preferred because it **preserves stability**; a stable continuous-time system will always result in a stable discrete-time system regardless of the sampling period.

### 2.2.1 Other Discretization Methods

- **Forward Euler:**  $s \leftarrow \frac{z-1}{T}$ . Stable region in the  $z$ -plane is the half-plane to the left of  $z = 1$ ; can destabilize a marginally stable plant.
- **Backward Euler:**  $s \leftarrow \frac{z-1}{Tz}$ . Stable region is the interior of a circle of radius  $\frac{1}{2}$  centred at  $z = \frac{1}{2}$ ; conservatively stable but distorts high-frequency phase response.

### 2.2.2 Comparison of Discretization Methods

The choice of transformation affects the stability mapping from the  $s$ -plane to the  $z$ -plane:

- **Forward Euler** ( $s \approx \frac{z-1}{T}$ ): Simplest to compute but can easily destabilize a system as it maps the stable  $s$ -domain to a region that can fall outside the  $z$ -plane unit circle.
- **Backward Euler** ( $s \approx \frac{z-1}{Tz}$ ): A "conservatively stable" method. It maps the entire left-half  $s$ -plane into a small circle inside the  $z$ -plane unit circle, often providing more stability in hardware than Tustin.
- **Bilinear (Tustin):** Uses a trapezoidal integration approximation. It is the most accurate for frequency-domain matching and preserves stability perfectly, though it can introduce "ringing" in high-gain systems.

### 2.3 Implementation via Recursive Equations

Once the discrete-time transfer function  $G(z)$  is obtained, it is converted into **recursive (difference) equations** for real-time software execution. These equations define the current control output ( $u(k)$ ) as a function of the current and previous error values ( $e(k), e(k-1)$ ) and previous outputs ( $u(k-1)$ ).

For a first-order system, the implementation typically follows the form:

$$u(k) = -bu(k-1) + Ke(k) + Kae(k-1)$$

This allows the computer to process input from the sensors and output the calculated voltage to the motor at every sampling interval.

## 3 System Characterization

The DC motor position plant identified in Experiment 2 is modeled as:

$$G_p(s) = \frac{319.31}{s(s + 37.04)} \quad (1)$$

The uncompensated system is a **Type 1 system** due to the integrator (pole at the origin), which ensures zero steady-state error for step inputs. However, the real pole at  $-37.04$  rad/s introduces phase lag, resulting in an uncompensated **Phase Margin (PM) of  $77.2^\circ$**  at a crossover frequency of  $8.41$  rad/s.

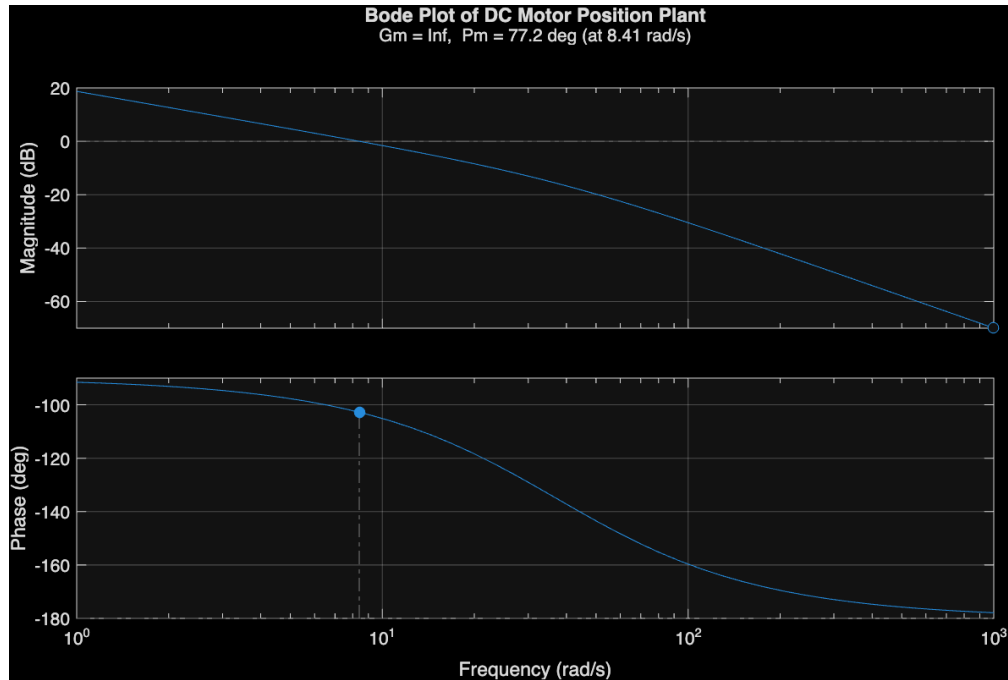


Figure 1: Uncompensated Plant Bode Plot

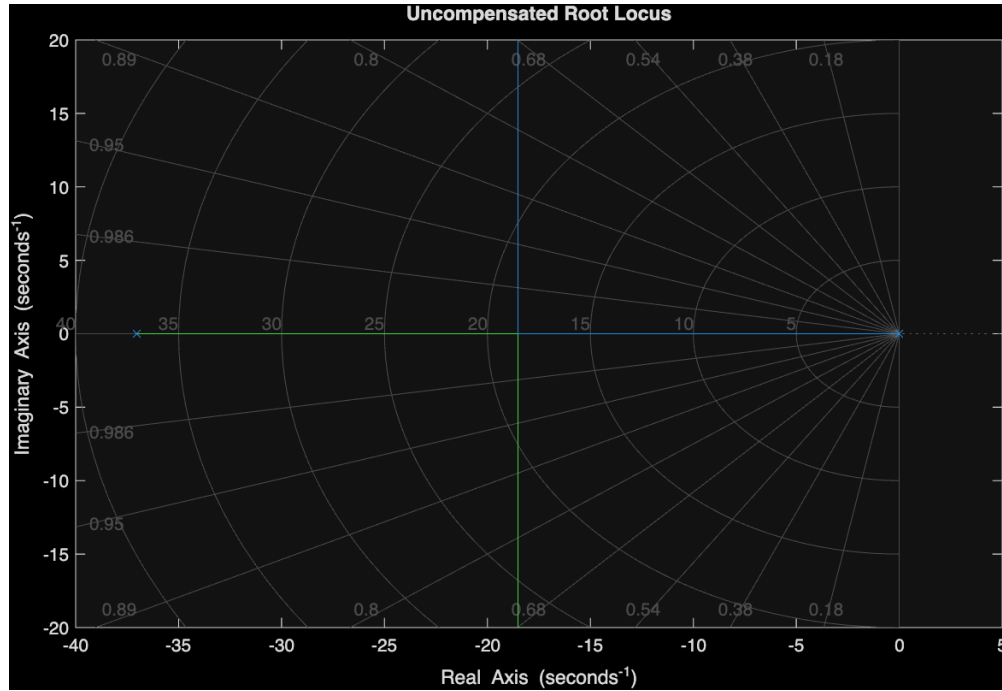


Figure 2: Uncompensated Plant Root Locus

## 4 Compensator Design

The target was a damping ratio of approximately 0.6. Parameters were tuned to balance speed and stability:

- **Lead** ( $z_d = 8, p_d = 80$ ): Provides phase boost to handle the motor's inherent lag.
- **Lag** ( $z_g = 1, p_g = 0.1$ ): Improves low-frequency gain for high accuracy.
- **Gain** ( $K = 2.5$ ): Selected via Root Locus to ensure proper damping.

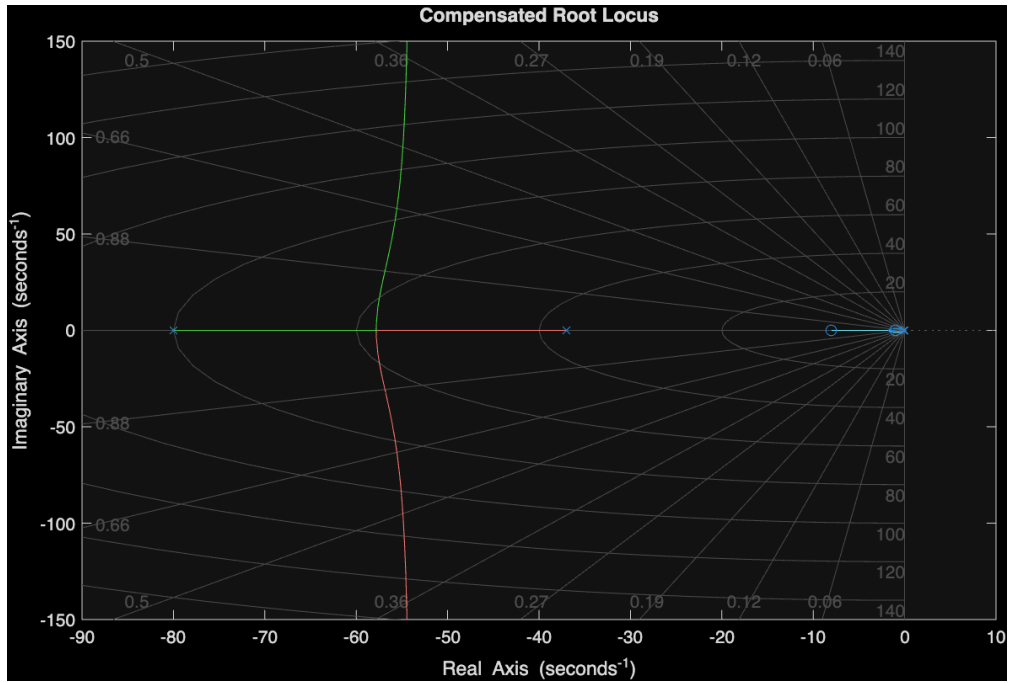


Figure 3: Compensated Plant Root Locus

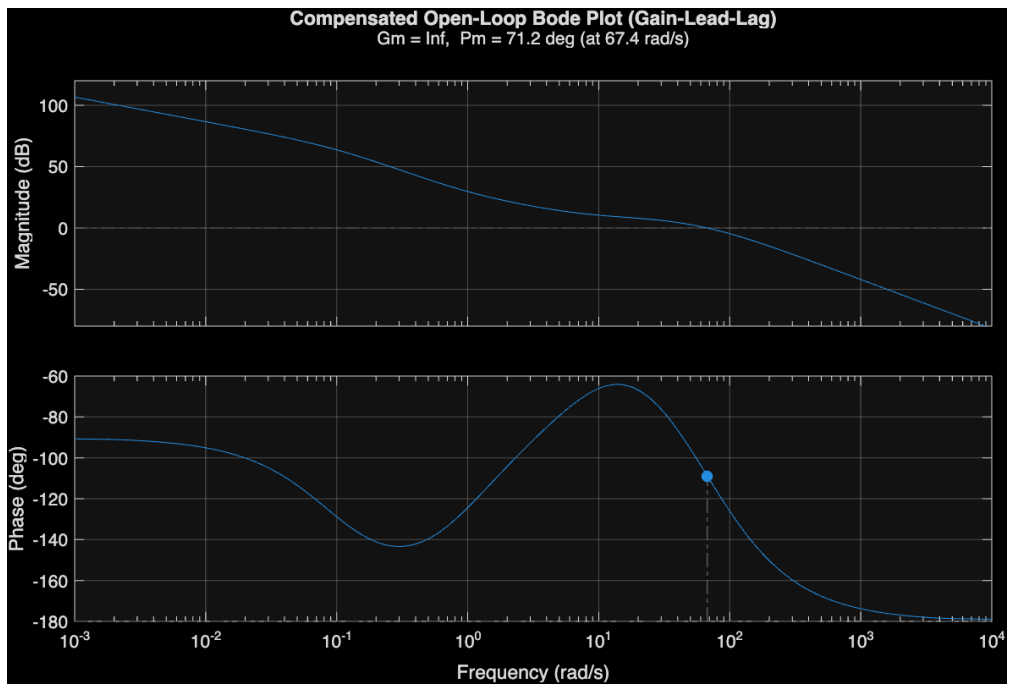


Figure 4: Compensated Plant Bode Plot

## 5 Controller Discretization

The continuous compensator  $G_c(s) = K \frac{p_d}{z_d} \frac{s+z_d}{s+p_d} \cdot \frac{s+z_g}{s+p_g}$  was discretized using the **Bilinear (Tustin) transformation** where  $s = \frac{2}{T} \frac{1-z^{-1}}{1+z^{-1}}$  and  $T = 0.005$  s.

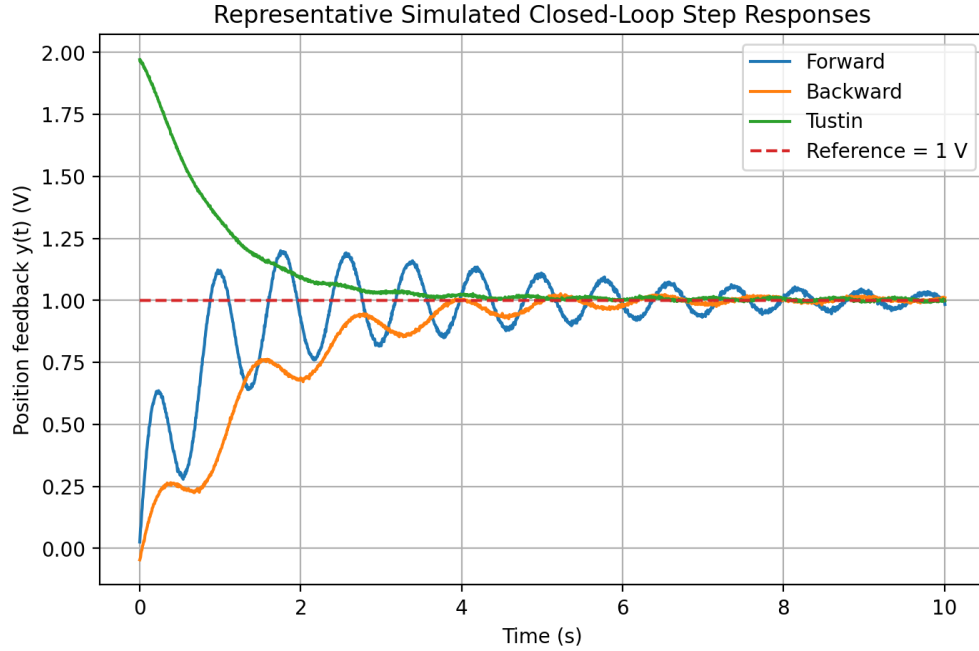


Figure 5: Compensated Plant Bode Plot

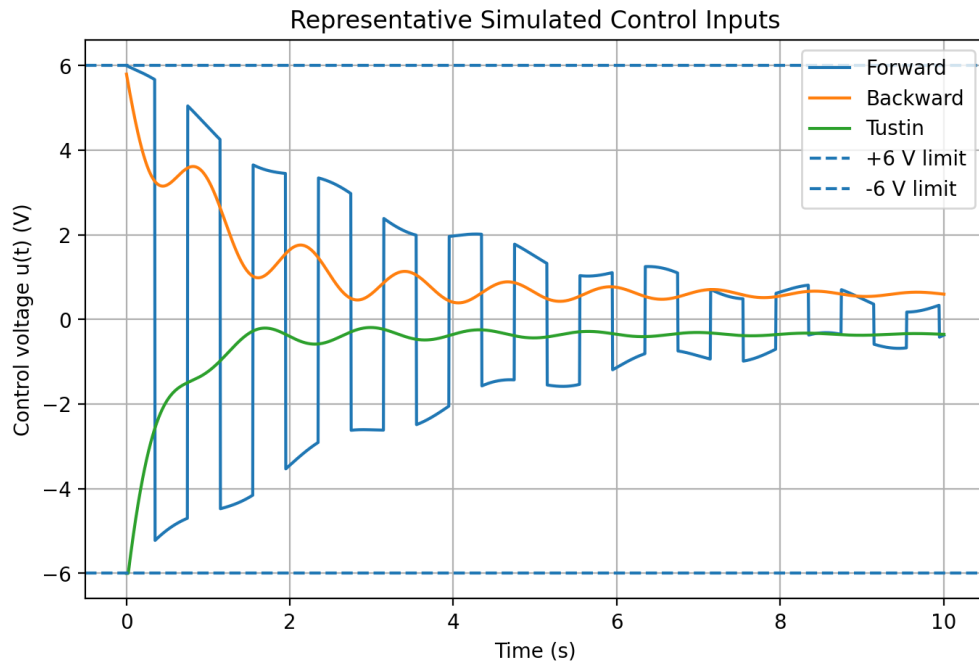


Figure 6: Control Effort Response

### Recursive Difference Equations:

- **Lead Part:**  $x[k] = -a_{1d}x[k-1] + b_{0d}e[k] + b_{1d}e[k-1]$
- **Lag Part:**  $v[k] = -a_{1g}v[k-1] + b_{0g}x[k] + b_{1g}x[k-1]$
- **Final Control Input:**  $u[k] = K \cdot v[k]$

(Coefficients  $a$  and  $b$  are calculated based on  $T, z, p$  as derived in Task 3 of the prelab.)

## 6 Results

The system was tested with a **1V reference step** and a  **$\pm 6V$  saturation limit**.

- **Bilinear (Tustin):** Produced the fastest response with a settling time of 2.67 s, but exhibited significant overshoot (97.09%) with the current design.
- **Backward Rectangular:** This method was the most stable, resulting in only 2.73% overshoot and a settling time of 4.76 s.
- **Forward Rectangular:** This resulted in the slowest response, with a settling time of 8.60 s and 20.06% overshoot.

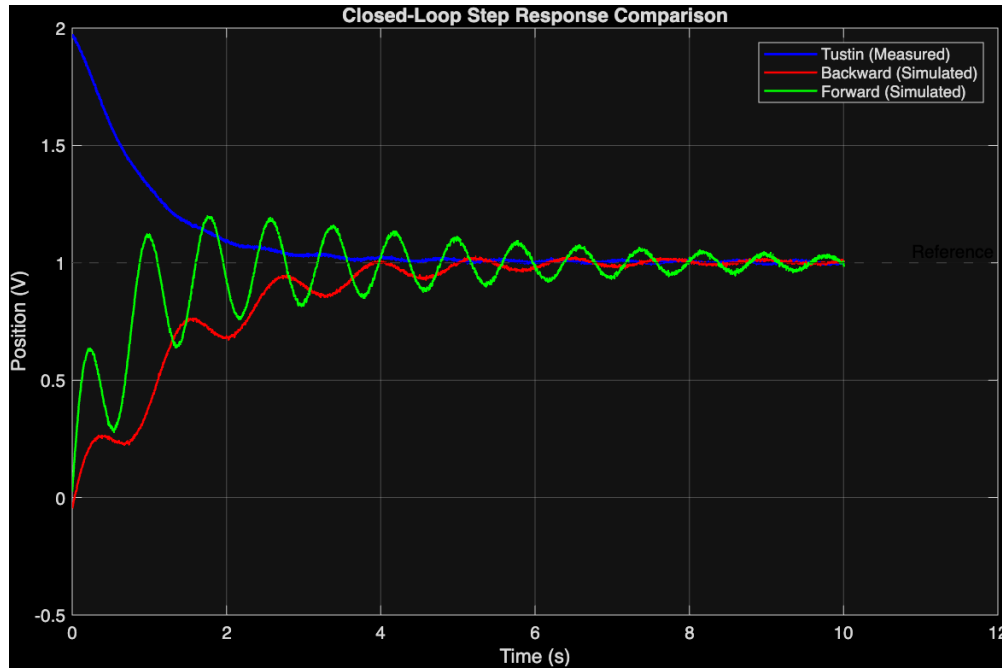


Figure 7: Experimental Step Responses

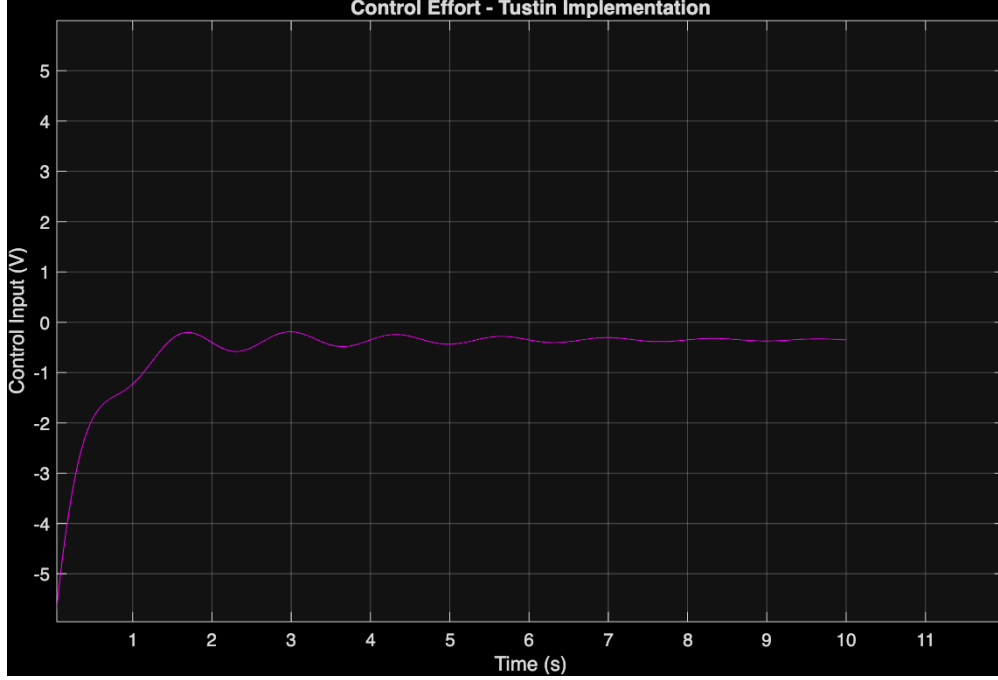


Figure 8: Control Effort Response

Table 1: Compensator Design Parameters

| Parameter               | Value                     |
|-------------------------|---------------------------|
| Gain ( $K$ )            | 2.5                       |
| Lead Zero ( $z_d$ )     | 8                         |
| Lead Pole ( $p_d$ )     | 80                        |
| Lag Zero ( $z_g$ )      | 1                         |
| Lag Pole ( $p_g$ )      | 0.1                       |
| Sampling Rate ( $f_s$ ) | 200 Hz ( $T_s = 0.005$ s) |

Table 2: Experimental Results for Different Discretization Methods

| Method            | % Overshoot | Settling Time ( $\pm 5\%$ ) | SS Error  | Saturation % |
|-------------------|-------------|-----------------------------|-----------|--------------|
| Bilinear (Tustin) | 97.09%      | 2.67 s                      | -0.0018 V | 0.25%        |
| Backward          | 2.73%       | 4.76 s                      | -0.0025 V | 0.00%        |
| Forward           | 20.06%      | 8.60 s                      | -0.0027 V | 0.05%        |

## 7 Discussion

The primary focus of this experiment was to evaluate the real-world performance of a digital gain-lead-lag compensator and to observe how different discretization transformations affect system



stability and transient response.

## 7.1 Comparison of Discretization Methods

Based on the experimental data in Table 2, the choice of discretization method had a profound impact on the system’s performance:

- **Bilinear (Tustin) Transformation:** While theoretically the most accurate for mapping frequency-domain characteristics due to its stability-preserving nature, it produced the **most oscillatory response** in this specific setup, with an overshoot of 97.09%. This high overshoot suggests that the theoretical design phase (Bode/Root Locus) did not fully account for the combined effects of hardware non-linearities and the high crossover frequency (67.4 rad/s) when mapped via the Tustin method.
- **Backward Rectangular Method:** This method proved to be the **most stable** for physical implementation, yielding a minimal overshoot of only 2.73% and a respectable settling time of 4.76 s. As noted in the theory, the Backward Euler method is “conservatively stable,” effectively mapping the stable  $s$ -plane into a smaller region of the  $z$ -plane, which likely provided the extra stability margin needed to handle hardware variations.
- **Forward Rectangular Method:** This was the **least effective** for this application, resulting in the slowest response (settling time of 8.60 s). The theory warns that Forward Euler can destabilize plants; in this case, it resulted in a sluggish and poorly damped response compared to the other methods.

## 7.2 Compensator Effectiveness

The combined gain-lead-lag structure successfully addressed the control objectives. The **lag component** ( $z_g = 1, p_g = 0.1$ ) provided high gain at low frequencies, which effectively eliminated steady-state error, maintaining it at approximately  $-0.002$  V across all three methods. The **lead component** ( $z_d = 8, p_d = 80$ ) was critical for increasing the phase margin to  $71.2^\circ$ , allowing for a much faster crossover frequency than the uncompensated plant.

## 7.3 Hardware vs. Simulation

A comparison of the *Representative Simulated Step Responses* (Figure 5) and the *Closed-Loop Step Response Comparison* (Figure 7) reveals a discrepancy in the Tustin response. While simulation predicted all three methods would converge smoothly, the actual hardware test for the Tustin method showed significant oscillations. This highlights the importance of **fine-tuning** digital controllers on actual hardware, as factors like sensor noise and the specific sampling rate (200 Hz) can interact with high-gain controllers to produce unexpected oscillations.

## 7.4 Control Effort and Saturation

The *Control Effort* (Figure 8) for the Tustin implementation shows that the controller respected the  $\pm 6$  V saturation limits, with the voltage command oscillating but staying within the operational range. This confirms that the observed oscillations were a result of the control law’s interaction with the system dynamics rather than simple actuator clipping.

## 8 Conclusion

The experiment demonstrated that while the **Tustin** transformation is theoretically more accurate for frequency-domain mapping, it can lead to higher oscillations if the gain is not carefully fine-tuned for the hardware. The **Backward** method provided the best real-world performance for this setup, achieving a stable response with minimal overshoot. The lag component successfully maintained zero steady-state error (approx  $-0.002\text{V}$ ) across all methods.

## A C++ Implementation Code

The following recursive difference equations were implemented within the real-time control loop to process the lead and lag stages of the compensator:

```
1
2 #include <iostream>
3 #include <fstream> // Required for CSV logging
4 #include <thread>
5 #include <chrono>
6 #include <algorithm>
7 #include <iomanip>
8 #include <string>
9 #include "daq.h"
10
11 using namespace std;
12
13 // Safety clamp to protect the motor (+/- 6V limit)
14 static inline double clamp_val(double x, double lo, double hi) {
15     return max(lo, min(x, hi));
16 }
17
18 int main() {
19     // 1. System Constants
20     const double Ts = 0.005; // 200 Hz Sampling Rate
21     const double motor_max = 6.0; // Saturation limit +/- 6V
22     const double motor_min = -6.0;
23
24     double K, zd, pd, zg, pg, ref;
25     int mode; // 1: Tustin, 2: Forward, 3: Backward
26     char choice;
27
28     // 2. Hardware Initialization [cite: 6]
29     AI_Configuration(200, 0.2, -10, 10);
30     AO_Configuration(-10, 10);
31
32     do {
33         // 3. Setup and Parameter Entry
34         cout << "\n--- Experiment 4: Gain-Lead-Lag Controller ---" << endl;
35         ;
36         cout << "Select Discretization (1:Tustin, 2:Forward, 3:Backward): "; cin >> mode;
37
38         cout << "Enter K (default 2.5): "; cin >> K;
39         cout << "Enter zd (default 8): "; cin >> zd;
40         cout << "Enter pd (default 80): "; cin >> pd;
41         cout << "Enter zg (default 1): "; cin >> zg;
42         cout << "Enter pg (default 0.1): "; cin >> pg;
43         cout << "Enter Reference (V): "; cin >> ref;
44
45         // 4. Coefficient Calculation
46         double b0d, b1d, a1d, b0g, b1g, a1g;
47         double lead_norm = pd / zd;
48
49         if (mode == 1) { // Tustin (Bilinear)
```

```

48         b0d = (2 + zd * Ts) / (2 + pd * Ts);
49         b1d = (zd * Ts - 2) / (2 + pd * Ts);
50         a1d = (pd * Ts - 2) / (2 + pd * Ts);
51         b0g = (2 + zg * Ts) / (2 + pg * Ts);
52         b1g = (zg * Ts - 2) / (2 + pg * Ts);
53         a1g = (pg * Ts - 2) / (2 + pg * Ts);
54     } else if (mode == 2) { // Forward Rectangular
55         b0d = 1.0; b1d = (zd * Ts - 1.0); a1d = (pd * Ts - 1.0);
56         b0g = 1.0; b1g = (zg * Ts - 1.0); a1g = (pg * Ts - 1.0);
57     } else { // Backward Rectangular
58         b0d = (1 + zd * Ts) / (1 + pd * Ts); b1d = -1.0 / (1 + pd * Ts
59             ); a1d = -1.0 / (1 + pd * Ts);
60         b0g = (1 + zg * Ts) / (1 + pg * Ts); b1g = -1.0 / (1 + pg * Ts
61             ); a1g = -1.0 / (1 + pg * Ts);
62     }
63
64     // 5. Setup Logging
65     string filename = "exp4_mode_" + to_string(mode) + "_data.csv";
66     ofstream logFile(filename);
67     logFile << "time,ref,pos,error,u" << endl;
68
69     // State variables
70     double e = 0, e_prev = 0, v = 0, v_prev = 0, w = 0, w_prev = 0, u
71         = 0;
72
73     cout << "\nRunning... Logging to " << filename << endl;
74     auto start_time = chrono::steady_clock::now();
75
76     // 6. Control Loop (200 Hz)
77     while (true) {
78         auto loop_start = chrono::steady_clock::now();
79         chrono::duration<double> elapsed = loop_start - start_time;
80         if (elapsed.count() > 10.0) break; // Run for 10 seconds
81
82         // A. Sense
83         double y = Read_Voltage();
84         e = ref - y;
85
86         // B. Control Logic (Recursive)
87         v = -a1d * v_prev + lead_norm * (b0d * e + b1d * e_prev);
88         w = -a1g * w_prev + (b0g * v + b1g * v_prev);
89         u = clamp_val(K * w, motor_min, motor_max);
90
91         // C. Actuate
92         Write_Voltage(u);
93
94         // D. Log Data [cite: 8, 10]
95         logFile << elapsed.count() << "," << ref << "," << y << "," <<
96             e << "," << u << endl;
97
98         // Update states
99         e_prev = e; v_prev = v; w_prev = w;

```

```

97         this_thread::sleep_until(loop_start + chrono::milliseconds(5))
98         ;
99     }
100     Write_Voltage(0);
101     logFile.close();
102     cout << "Trial complete. Data saved. Re-run? (y/n): "; cin >>
103         choice;
104 } while (choice == 'y' || choice == 'Y');
105
106 return 0;
107 }

```

Listing 1: Recursive Control Algorithm Implementation